Data Wrangling

CMPSC 105 – Data Exploration

ALLEGHENY COLLEGE

Goals

- Define the data wrangling cycle
- Review sanity checks on data
- Review common operations to manipulate data
- Use pandas to wrangle data in notebook

Data Wrangling Cycle

- Data wrangling is the process of cleaning, preprocessing and validating raw data before use

Data Wrangling Cycle

Name, Age, Salary
Alice, 23, $50k
Bob, NA, $60k
Chris, 30, sixty thousand

Can we analyze this directly?

Cycle


Discovering
Structuring
Cleaning
Enriching
Validating
Publishing
Data Wrangling

Sanity Checks on Data

- Shape –> df.shape()
- Column names –> df.columns
- Types of data –> df.dtypes

If a column has a mix of strings and other data types (e.g., numbers or None values), pandas defaults to object. [GPT]

Common Operations

Accessing specific rows and columns

- loc –> df.loc[rows, cols]
- iloc –> df.iloc[rows, cols by index]

Creating a new column in an existing df

- df[‘new’] = expression
- Note that a new column can only be added if it is the same length as the other existing columns

Common Operations

Concatenation

- Along Rows –> axis=0
- Along Cols –> axis=1
- pd.concat(df1, df2, axis=1)

Along Rows (axis = 0)

DataFrame 1

name	age
Alice	20
Bob	21

DataFrame 2

name	age
Chris	22
David	23

pd.concat([df1, df2], axis=0)

name	age
Alice	20
Bob	21
Chris	22
David	23

Along Columns (axis = 1)

DataFrame 1

name
Alice
Bob

DataFrame 2

age
20
21

pd.concat([df1, df2], axis=1)

name	age
Alice	20
Bob	21

Common Operations

Reformatting Strings

- Regular Expressions inside of pd.DataFrame.str.replace()
  - [] Groups together characters to search for
  - [^...] Negation (matches anything not inside the brackets).
  - a-z Matches lowercase letters a to z.
  - + Matches one or more occurrences of the unwanted characters.

Common Operations

Reformatting Strings

text
apple!!!
banana123
orange@@

remove non-letter characters

```
df["text"] = df["text"].str.replace("[^a-z]+", "", regex=True)
```

Common Operations

Reformatting Strings

- df[‘YOUR COLUMN’].str.split(‘YOUR DELIMITER’, expand=True)

location
Pittsburgh, PA
New York, NY
Boston, MA

df[["city","state"]] = df["location"].str.split(",", expand=True)

Activity

Data Wrangling Activity

- data_wrangling_activity.ipynb